Scikit-learn (sklearn)

三、核心模块

1. 监督学习算法

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 分类算法
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# 回归算法
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

2. 无监督学习算法

```
# 聚类
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering

# 降维
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

3. 数据预处理

```
from sklearn.preprocessing import (
    StandardScaler,      # 标准化
    MinMaxScaler,        # 归一化
    LabelEncoder,        # 标签编码
    OneHotEncoder        # 独热编码
)

from sklearn.impute import SimpleImputer  # 缺失值处理
```

4. 模型选择与评估

```
from sklearn.model_selection import (
    train_test_split,    # 数据集划分
    cross_val_score,     # 交叉验证
    GridSearchCV,        # 网格搜索
    RandomizedSearchCV   # 随机搜索
)

from sklearn.metrics import (
    accuracy_score,      # 准确率
    precision_score,     # 精确率
    recall_score,        # 召回率
    f1_score,           # F1分数
    confusion_matrix,    # 混淆矩阵
    roc_auc_score       # AUC值
)
```

四、典型工作流程

完整示例：分类任务

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1. 加载数据
iris = load_iris()
X, y = iris.data, iris.target

# 2. 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. 数据预处理
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. 训练模型
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)

# 5. 预测
y_pred = model.predict(X_test_scaled)

# 6. 评估
print("混淆矩阵:\n", confusion_matrix(y_test, y_pred))
print("\n分类报告:\n", classification_report(y_test, y_pred))
print("准确率:", model.score(X_test_scaled, y_test))
```

回归任务示例

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 生成数据
X, y = make_regression(n_samples=100, n_features=1, noise=10)

# 划分数据
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 训练
model = LinearRegression()
model.fit(X_train, y_train)

# 预测
y_pred = model.predict(X_test)

# 评估
print("MSE:", mean_squared_error(y_test, y_pred))
print("R²:", r2_score(y_test, y_pred))
```

五、高级功能

1. 交叉验证

```
from sklearn.model_selection import cross_val_score

model = RandomForestClassifier()
scores = cross_val_score(model, X, y, cv=5)

print("交叉验证分数:", scores)
print("平均分数:", scores.mean())
```

六、常用算法对比

算法类型	适用场景	优点	缺点
逻辑回归	二分类	简单快速	线性假设
决策树	分类/回归	可解释性强	易过拟合
随机森林	分类/回归	准确率高	训练慢
SVM	高维数据	效果好	大数据集慢
KNN	小数据集	无需训练	预测慢